FedProp: Federated Graph Neural Networks with Feature Propagation

1st Brian Kipkirui
Carnegie Mellon University Africa
Kigali, Rwanda
bkipkiru@andrew.cmu.edu

2nd João Barros
Electrical and Computer Engineering
Carnegie Mellon University Africa
Kigali, Rwanda
jbarros@andrew.cmu.edu

***Abstract*—Graph Neural Networks (GNNs) are a powerful tool to learn from relational data; however, training them in federated, privacy-preserving settings still poses considerable challenges. A key obstacle is the presence of so-called cross-client edges, i.e., edges whose incident nodes reside on distinct devices. Because standard message-passing GNNs aggregate features across neighboring nodes, these cross-client edges lead to information gaps that degrade accuracy when communication between client devices is disallowed. We recast this obstacle as a missing-feature problem and introduce FedProp: a model-agnostic federated GNN framework that reconstructs the l-hop features required for aggregation via feature propagation. In contrast to prior approaches, which either learn additional models to generate the missing neighbors or communicate the missing information, FedProp offers a lightweight iterative process that runs locally, requiring no inter-client communication or additional parameters. First, we provide a theoretical justification for feature propagation in subgraphs and provide a convergence guarantee of the iterative process. We then empirically demonstrate that FedProp can significantly improve GNN performance. For example, on Cora (Non-IID) with a GCN backbone, FedProp achieves 81.0% accuracy, a 19.1% relative improvement over the federated baseline (0.680). FedProp thus enables users to deploy powerful GNNs across various client devices with locally stored data, achieving near-optimal accuracy and minimal communication costs.**

***Index Terms*—Graph Neural Networks, GNNS, Subgraph Federated Learning, Feature Propagation**

I. Introduction

GRAPH Neural Networks (GNNs) are powerful tools for learning from graph-structured data [1]. Operating via a message-passing mechanism where nodes aggregate local neighborhood information to richer representations [2], GNNs have achieved impressive performance across diverse applications, including social networks, drug discovery, and recommender systems [3].

However, training GNNs on real-world graphs often faces a critical challenge: data is frequently decentralized and subject to privacy regulations like GDPR, necessitating a move towards frameworks like Federated Learning (FL) [4], [5]. This paper addresses a practical and increasingly common setting known as **'Subgraph-FL'** [6], where clients collaboratively train a model on their local subgraphs, which are partitions of a single, larger global graph. The 'Subgraph-FL' setting is applicable in a range of applications. In financial fraud detection, for instance, a transaction graph may be partitioned across different banks, yet a fraudulent transaction ring could span multiple institutions. Similarly, in recommender systems, user-item interaction graphs might be partitioned by region, but user preferences can have global patterns.

This research was supported by the Mastercard Foundation and the Upanzi Network at Carnegie Mellon University Africa.

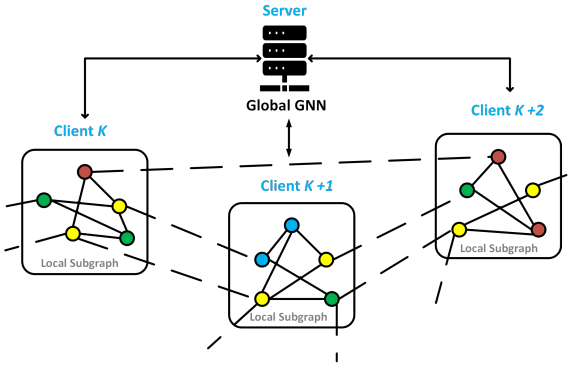

Fig. 1. Cross-client edges (dashed lines) connect nodes on distinct clients, leading to missing neighbor information that degrades GNN message passing.

In this setting, the use of graph neural networks becomes quite challenging. This is due to the fact that (i) each client only sees a subset of the nodes of the underlying graph and (ii) clients are only allowed to exchange model updates [7], [8] without taking into consideration the cross-client edges that connect graph nodes in distinct client devices (see Figure 1).

Such cross-client edges occur frequently in large classes of partitioned graphs (e.g. social networks or transportation networks). FL solutions that ignore them are prone to lose key information with consequent degradation of overall GNN performance, particularly in semi-supervised node classification. For example, when compared to a centralized solution [9], local training on Ogbn-Arxiv and Cora without accounting for cross-client connectivity resulted in a 14.6% and a 19.4% drop in accuracy, respectively. It is conceivable that clients could generate synthetic neighbors for their subgraphs or share partial information with other clients. However, those approaches incur significant communication overhead, require additional learnable parameters, or fail at ensuring the necessary level of privacy. This motivates the need for more effective federated GNN methodologies.

To overcome these challenges, we introduce FedProp, a model-agnostic framework that recasts the missing neighbor problem as a local feature imputation task. Guided by Dirichlet energy minimization, FedProp uses iterative propagation to reconstruct the k-hop neighborhood locally, requiring zero additional inter-client communication. We provide a formal convergence guarantee for this process and demonstrate empirically that it achieves near-centralized accuracy—recovering a high percentage of the baseline performance gap —while being orders of magnitude more communication-efficient than existing methods.

The remainder of the paper is organized as follows. Section 2 reviews related work, Section 3 provides fundamental background, and Section 4 presents the proposed FedProp methodology, including problem definition, algorithm, and convergence analysis. Section 5 discusses the experimental setup and presents the results. The paper ends with some conclusions, summarized in Section 6.

II. Related Work

GNNs are a class of machine learning models designed to learn from graph-structured data by aggregating information from neighboring nodes in the underlying graph. Popular GNNs models include GCN [10], GAT [11], and GraphSAGE [12]. In federated learning settings, where data is distributed across multiple clients and cannot be centralized due to privacy or regulatory restrictions, neighborhood aggregation becomes a significant challenge [13]. As a result, the best-performing GNN models in centralized environments may not perform equally well in federated scenarios.

Federated GNN methods address the missing neighbor problem in partitioned graphs via two main strategies. **Generative approaches**, such as FedSage+ [14] and FedNI [4], use local generative models (e.g., GANs) to impute missing neighbors or features, but introduce significant computational and communication overhead. *Transmission-based approaches*, including FedGCN [9] and FedGAT [15], share required neighbor features in a pre-training round and employ polynomial approximations (for GAT). However, they require extra communication and encryption, while being limited to specific GNN architectures.

FedProp differs fundamentally from the aforementioned methodologies in several ways: (a) FedProp is model-agnostic, (b) requires no extra inter-client communication, and (c) reconstructs missing features locally via feature propagation. Unlike other federated GNN methods, such as FedGCN, FedGAT, and FedSage+, which face significant trade-offs in privacy, communication overhead, and computational cost, FedProp fully preserves privacy with low communication, while maintaining broad compatibility with any existing message-passing GNN.

III. Fundamentals

In a federated setup, the graph G is partitioned across K clients with subgraphs G_k. Each node $v \in V_k$ has known neighbors $N_{\mathcal{K}}(v) = N(v) \cap V_k$ and unknown neighbors $N_{\mathcal{U}}(v) = N(v) \setminus V_k$ (connected through cross-client edges). Without cross-client communication, GNNs on client k can only aggregate from $N_{\mathcal{K}}(v)$, leading to significant performance degradation. FedProp addresses this by reconstructing initial features $\tilde{h}_u^{(\ell)}$ for unknown neighbors, enabling the new GNN update rule:

$$\mathrm{h}_v^{(\ell+1)} = f\Big(h_v^{(\ell)}, \{h_u^{(\ell)} : u \in N_{\mathcal{K}}(v)\}, \{\tilde{h}_u^{(\ell)} : u \in N_{\mathcal{U}}(v)\}, x_v\Big).$$

Our imputation method is guided by the principle of feature smoothness, quantified by the **Dirichlet energy** $\mathcal{E}(X)$. For a node-feature matrix X and normalized Laplacian L, this is:

$$\mathcal{E}(X) = \mathrm{tr}(X^\top L X) = \frac{1}{2} \sum_{(i,j)\in E} A_{ij} \left\| \widetilde{X}_i - \widetilde{X}_j \right\|_2^2, \qquad (1)$$

where low energy indicates similar features across neighbors. **Node feature propagation** is an iterative process that minimizes this energy subject to known boundary constraints. It uses a diffusion operator $\widehat{P}$ ($X^{(t+1)} = \widehat{P}\, X^{(t)}$) to diffuse known information, effectively filling in missing values based on graph connectivity and smoothness [16]. Standard operators $\widehat{P}$ include normalized adjacency, Personalized PageRank [17], and the Heat Kernel.

IV. The FedProp Method

As established in the Introduction, we address the challenge of federated learning in the **'Subgraph-FL' setting**. In this scenario, a global graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ is divided between K clients, where each client k has a local subgraph $\mathcal{G}_k = (\mathcal{V}_k, \mathcal{E}_k)$ and its corresponding node features $\mathbf{X}_k$. The core challenge comes from the message-passing mechanism of GNNs. For a node $v \in \mathcal{V}_k$ to compute its updated representation, it must aggregate information from its entire neighborhood, $\mathcal{N}(v)$. However, in a standard federated setup, client k only possesses features for nodes within its own partition, $\mathcal{V}_k$. It has no access to the features of any remote neighbors—that is, any node $u \in \mathcal{N}(v) \setminus \mathcal{V}_k$. This prevents local nodes from aggregating a complete neighborhood representation, leading to the information gap illustrated by the cross-client edges in Figure 1 and causing a significant degradation in GNN performance.

We address this missing neighbor problem by reformulating it as a **feature imputation task**. Our proposed method, FedProp, enables each client to locally and efficiently estimate the unknown features of its remote neighbors. This enables the reconstruction of a complete 1-hop neighborhood for message passing, allowing for full-graph GNN updates without violating privacy constraints. The process is illustrated in Figure 2.

To perform feature imputation, each client first constructs an **accessible subgraph**, $G_k^{(L)} = (V_k^{(L)}, E_k')$, extending up to L hops from its local nodes V_k. The precise structure of this subgraph depends on the assumed privacy model. In our experiments, we evaluate two common scenarios: a **Strict Privacy Model** ($L = 1$) where clients have no central support, and a model with a **centrally known topology** ($L = 2$), a

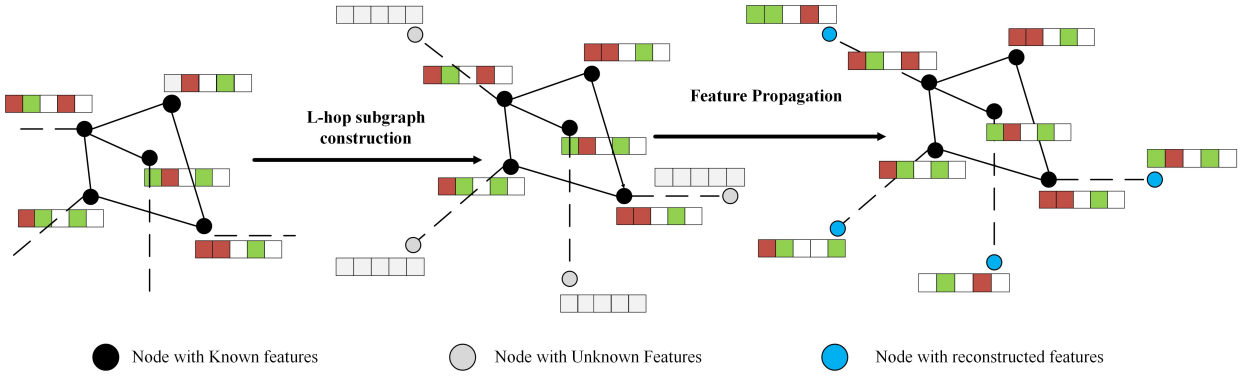

Fig. 2. **The FedProp Feature Propagation Process.** The diagram visualizes the FedProp algorithm, which reconstructs missing features for an L-hop subgraph from the initial subgraph. The process involves iteratively propagating features to fill in missing information, allowing for a complete view of the L-hop subgraph.

practical setting for many real-world applications[1] We evaluate both cases, as performance gains were observed to diminish for $L > 2$, consistent with prior work [9].

a) Feature Matrix Initialization.: For feature propagation, an augmented feature matrix $\mathbf{X}_k^{(0)}$ is constructed for client k's accessible subgraph, where known local node features ($\mathbf{x}_i$ for $i \in V_k$) are retained, and features for unknown remote nodes ($i \in V_k^{(L)} \setminus V_k$) are initialized to zeros.

b) Imputation via Dirichlet Energy Minimization.: Given the accessible subgraph $G_k^{(L)}$, our goal is to impute the unknown remote features $\mathbf{X}_{U_k}$. We formulate this as an energy minimization problem guided by the principle of **feature smoothness**. This smoothness is measured by the **local Dirichlet energy**, $\mathcal{E}_k(\mathbf{X}_k) = \frac{1}{2}\mathrm{tr}(\mathbf{X}_k^\top L_k \mathbf{X}_k)$. Our objective is to find the feature matrix $\mathbf{X}_k^*$ that **minimizes this energy value**, subject to the constraint that the known local features $\mathbf{X}_{V_k}$ remain fixed.

The minimum is found at the steady state where the gradient of the energy with respect to the unknown features is zero: $(\nabla_{\mathbf{X}_{U_k}} \mathcal{E}_k)(\mathbf{X}_k^*) = (L_k \mathbf{X}_k^*)_{U_k} = 0$. To analyze this condition, we partition the Laplacian L_k and feature matrix $\mathbf{X}_k$ into blocks corresponding to known (V_k) and unknown (U_k) nodes:

$$L_k = \begin{bmatrix} L_{V_kV_k} & L_{V_kU_k} \\ L_{U_kV_k} & L_{U_kU_k} \end{bmatrix}, \quad \mathbf{X}_k = \begin{bmatrix} \mathbf{X}_{V_k} \\ \mathbf{X}_{U_k} \end{bmatrix}.$$

The steady-state condition $(L_k \mathbf{X}_k^*)_{U_k} = 0$ thus expands to $L_{U_kV_k}\mathbf{X}_{V_k} + L_{U_kU_k}\mathbf{X}_{U_k}^* = 0$. Solving for $\mathbf{X}_{U_k}^*$ yields the unique analytical solution, which we formalize below.

Proposition 1 (Analytical Solution)**.** *For a connected accessible subgraph $G_k^{(L)}$, the unique feature matrix $\mathbf{X}_{U_k}^*$ that minimizes the Dirichlet energy is given by:*

$$\mathbf{X}_{U_k}^* = -L_{U_kU_k}^{-1} L_{U_kV_k} \mathbf{X}_{V_k}. \tag{2}$$

[1]The subgraph construction differs by model. Under Strict Privacy (L=1), clients use only local data, meaning remote neighbors appear as leaves without knowledge of their interconnections. In contrast, for L=2, we assume a common setting where a server provides the complete 2-hop topology, crucially including any edges between remote nodes [13]. Both models preserve FedProp's core guarantee of zero communication during training.

Proof. A detailed derivation is provided in the Supplementary material.

While this closed-form solution exists, directly computing the matrix inverse is computationally prohibitive, which motivates our efficient, iterative approach.

c) Iterative Solution via Discretization and Boundary Reset.: To find the steady state solution, we approximate the solution by discretizing the analytical solution in equation Eq. (2) using the explicit Euler method with a small step size $h > 0$. This yields the iterative update:

$$\mathbf{X}_k^{(t+1)} = \mathbf{X}_k^{(t)} - hL_k\mathbf{X}_k^{(t)}.$$

By setting $P_k := I - hL_k$, this iteration simplifies to $\mathbf{X}_k^{(t+1)} = P_k\mathbf{X}_k^{(t)}$. To enforce the Dirichlet boundary condition that the features of known local nodes 'V_k' remain fixed at their initial values '$\mathbf{X}_{V_k}^0$', we reset these features at each iteration. The full iterative update with boundary reset is thus:

$$\mathbf{X}_k^{(t+1)} \leftarrow P_k\mathbf{X}_k^{(t)}, \text{then } \mathbf{X}_k^{(t+1)}(v) = \mathbf{X}_{V_k}^0(v), \forall v \in V_k. \tag{3}$$

To show explicitly how this impacts the unknown nodes, we partition the propagation matrix P_k and the feature matrix $\mathbf{X}_k^{(t)}$ according to known and unknown nodes:

$$P_k = \begin{bmatrix} P_{V_kV_k} & P_{V_kU_k} \\ P_{U_kV_k} & P_{U_kU_k} \end{bmatrix}, \quad \mathbf{X}_k^{(t)} = \begin{bmatrix} \mathbf{X}_{V_k}^0 \\ \mathbf{X}_{U_k}^{(t)} \end{bmatrix}.$$

Applying the update rule and enforcing the fixed '$\mathbf{X}_{V_k}^0$', the iteration for the unknown features $\mathbf{X}_{U_k}$ becomes:

$$\mathbf{X}_{U_k}^{(t+1)} = P_{U_kU_k}\mathbf{X}_{U_k}^{(t)} + P_{U_kV_k}\mathbf{X}_{V_k}^0. \tag{4}$$

d) Convergence Guarantee.: The iterative process described by Eq. (3) is guaranteed to converge to the unique solution from Proposition 1. This is formally stated in Theorem 1.

Theorem 1 (Convergence of Feature Propagation)**.** *For any connected accessible subgraph $G_k^{(L)}$, if the propagation matrix P_k is chosen such that it is a spectral function of the Laplacian and its spectral radius $\rho(P_k) \leq 1$, then the iterative update*

in Eq. 4 is a contractive mapping. The spectral radius of the submatrix for unknown nodes, $\rho = \rho(P_{U_k U_k})$, is strictly less than one, guaranteeing the sequence $\mathbf{X}_{U_k}^{(t)}$ converges linearly to the unique solution $\mathbf{X}_{U_k}^*$.

Proof. (Sketch) Full proof is in the Appendix. The iteration (Eq. 4) is an affine recursion. It converges via the Banach fixed-point theorem because our boundary-reset on V_k anchors the subgraph, removing the non-contracting $\lambda = 1$ eigenvector from the U_k subspace. This ensures the operator $P_{U_k U_k}$ is a strict contraction ($\rho(P_{U_k U_k}) < 1$). We then show its fixed point equals the unique Dirichlet minimizer $X_{U_k}^*$ from Prop. 1.

Corollary 1 (Error Bound Analysis)**.** *The total reconstruction error, $\|\mathbf{X}_{U_k}^{(t)} - \mathbf{X}_{true,U_k}\|$, is governed by three factors. Our formal proof (see Appendix) uses the triangle inequality to introduce the iteration's fixed point $\mathbf{X}_{U_k}^*$ as an intermediate waypoint. This strategy splits the total error into:*

1) ***Propagation Error*** *($\|\mathbf{X}_{U_k}^{(t)} - \mathbf{X}_{U_k}^*\|_F$): A transient error from the iteration itself, which decays exponentially to zero.*
2) ***Stationary Error*** *($\|\mathbf{X}_{U_k}^* - \mathbf{X}_{true,U_k}\|_F$): A persistent error composed of initial boundary bias and irreducible graph heterophily.*

This core principle is illustrated by the simplified global case, whose bound is:

$$\underbrace{\lambda_{gap}^t \|(I - P_1)X^{(0)}\|_F}_{\text{Convergence Error}} + \underbrace{\|P_1(X^{(0)} - X_{true})\|_F}_{\text{Initial Bias}} + \underbrace{\|(I - P_1)X_{true}\|_F}_{\text{Irreducible Error}} \tag{5}$$

Our practical, client-level algorithm is a strict generalization of this principle, maintaining this same three-part structure.

In practice, we terminate the iterative propagation when the Frobenius norm of the feature change falls below a tolerance ε, i.e., $\|\mathbf{X}_k^{(t+1)} - \mathbf{X}_k^{(t)}\|_F < \varepsilon$.

e) Generalization and Propagation Operators.: The FedProp framework is generalizable to any propagation operator P_k that meets two conditions: its spectral radius must be bounded by one ($\rho(P_k) \leq 1$) to guarantee convergence, and it must be a spectral function of the graph Laplacian L to ensure it converges to the correct energy-minimizing state. The theoretical matrix P_k from our derivation can be instantiated as any operator meeting these criteria. In our experiments, we use two such operators: the **normalized adjacency matrix** ($\hat{A}$), which is a direct function of the normalized Laplacian ($L_{\text{norm}} = I - \hat{A}$), and the **diffusion kernel** ($\exp(-\lambda L)$), which we computationally approximate using a truncated Taylor series expansion. A formal outline of valid operators is provided in the supplementary material.

f) Algorithm.: The complete FedProp process is illustrated in Algorithm 1.

Algorithm 1 Residual-Minimizing Feature Propagation with Boundary Reset

Require: Accessible subgraph $G_k^{(L)} = (V_k^{(L)}, E_k')$, Propagation Matrix $P_k = I - hL_k$, known features $\mathbf{X}_{V_k}^0$, max iterations T, tolerance ε
Ensure: Completed feature matrix $\mathbf{X}_k$

```
Initialize X_k^(0) ← 0 on U_k; set X_k^(0)(v) ← X_{V_k}^0(v) for all v ∈ V_k
for t = 0 to T − 1 do
    X_k^(t+1) ← P_k X_k^(t)                          ▷ Propagate features
    for all v ∈ V_k do                               ▷ Reset boundary nodes
        X_k^(t+1)(v) ← X_{V_k}^0(v)
    end for
    if ‖X_k^(t+1) − X_k^(t)‖_F < ε then              ▷ Check for convergence
        break
    end if
end for
return X_k^(t+1)                                     ▷ Return propagated features
```

g) Local GNN Training and Advantages.: After feature propagation, each client trains a standard GNN (GNN_θ) using the imputed features $\mathbf{X}_k'$ and local adjacency A_k, minimizing cross-entropy loss over labeled nodes $\mathcal{L}_k \subset V_k$. This preprocessing step incurs no additional inter-client communication beyond standard federated model aggregation.

FedProp offers significant advantages:

- **Computational Efficiency:** Costs '$\mathcal{O}(T_{\text{prop}} \cdot |E_{G_k^{(L)}}| \cdot d)$' for sparse matrix-dense matrix multiplication, typically converging in 30-50 iterations without costly inversions.
- **Low Communication:** Matches FedAvg, with communication limited solely to periodic aggregation of model parameters '$\mathcal{O}(R \cdot |\theta|)$'. All feature propagation is local.
- **Strong Privacy:** Raw node features and labels remain strictly local; no intermediate representations are transmitted, inherently compatible with Differential Privacy.

A detailed comparison of FedProp's properties against other federated GNN methods, highlighting these advantages, is presented in the Appendix (see supplemental materials).

V. Experiments and Results

Having proven the convergence of our algorithm, we are now ready to present a detailed experimental evaluation of FedProp in terms of node classification accuracy and communications cost for both IID and non-IID partitions of the underlying graph.

A. Experimental Setup

Our experiments use the standard homophilic citation graph datasets: Cora, Citeseer, Pubmed, and Ogbn-Arxiv. Data are partitioned into client subgraphs using a label Dirichlet distribution, with concentration parameter $\beta = 10000$ for IID

settings and $\beta = 10$ or $\beta = 1$ for non-IID, to emulate varying degrees of the missing-neighbor challenge.

We demonstrate FedProp's backbone-agnostic nature by coupling it with a GCN [10] and a GAT [11]. We also perform a rigorous ablation study (detailed in the Appendix) on propagation operators (normalized adjacency vs. diffusion) and the impact of positional encodings (PE). Our ablations confirm FedProp is effective with both backbones and find that diffusion-based propagation consistently performs best. Furthermore, PE significantly improves performance, especially in non-IID settings. Therefore, we present GCN results in the main paper for clarity, and our primary variant, FedProp, refers to FedProp (Diff-PE).

We compare our method against several baselines: (i) **Centralized (Oracle)**: a GNN trained on the full, global graph; (ii) **FedProp-Full (Oracle)**: an ideal federated setting where clients have the true 1-hop remote features; and (iii) **FedProp-Zero (Baseline)**: the standard federated baseline that only trains on local subgraphs, ignoring all cross-client edges.

For our method, feature propagation is run for 80 iterations. Model architectures are adapted to each dataset: the GCN for Cora/Citeseer/Pubmed uses 2 layers, 16 hidden units, ReLU, and 0.5 dropout, while the GCN for Ogbn-Arxiv uses 3 layers and 256 hidden units. Our GAT model (e.g., for Pubmed) uses 8 heads. For training, we use SGD for Cora/Citeseer (lr 0.5, weight decay 5×10^{-4}) and Adam for Pubmed and OGBN-Arxiv with same decay and learning rate of 0.01 and 0.005 respectively. All federated experiments use FedAvg and are trained for up to 200 epochs with early stopping (patience 20) on validation loss.

B. Communication efficiency

We demonstrate that **FedProp** achieves high accuracy and high communication efficiency.

Figure 3 visualizes this trade-off across 3 planetoid datasets and partition settings. We plot node classification accuracy against **Communication Efficiency**, defined as the inverse of communication cost. We establish a baseline cost of 1.0 for standard fedAdvg aggregation (which FedProp and DistGAT use), while other methods incur costs orders of magnitude higher. The ideal method, combining high accuracy and high efficiency, appears in the **top-right quadrant**.

As shown in Figure 3, our **FedProp** method consistently occupies this ideal top-right quadrant. It proves that our local-only propagation achieves accuracy that is highly competitive with (and often superior to) transmission-based methods like **FedGAT**, while remaining **orders of magnitude** more efficient.

C. Overall Node Classification Accuracy

We evaluate our framework on Cora, Citeseer, Pubmed, and Ogbn-Arxiv (IID/Non-IID). Table I first confirms the severe performance drop when naively training on local subgraphs (**FedProp-Zero**) compared to an oracle with full 1-hop features (**FedProp-Full**)—for example, 0.680 vs. 0.810 on Cora Non-IID.

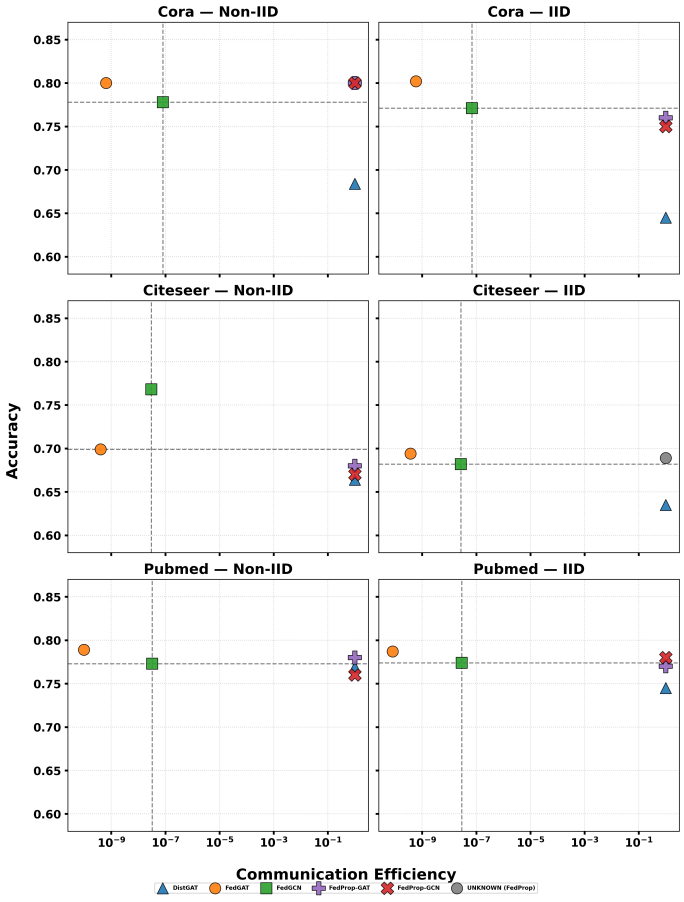

Fig. 3. Accuracy vs. Communication Efficiency trade-off across all datasets and partitions. **FedProp** consistently occupies the ideal top-right quadrant (high accuracy, high efficiency) [15].

Our method, FedProp-Diff, is designed to close this gap. The 1-hop variant (FedProp-Diff (1-hop)) already proves highly effective, recovering a substantial portion of lost performance and achieving near-oracle results in challenging Non-IID settings, such as 92.3% recovery on Cora and 140.0% on Ogbn-Arxiv, where the smoothing may act as a beneficial regularizer.

We can further enhance this strong baseline with 2-hop propagation. The FedProp-Diff (2-hop) variant delivers even more consistent, high-end accuracy, especially in the IID setting where 1-hop struggles. For instance, 2-hop boosts the IID recovery on Cora from 68.4% to 78.9% and on Citeseer from 55.6% to 66.7%. This refinement achieves state-of-the-art recovery, notably 100.0% of the gap on Cora (Non-IID). Crucially, FedProp achieves this robust accuracy at zero additional communication cost, making it ideal for privacy-sensitive environments.

VI. Conclusion and Future Work

We introduced **FedProp**, a model-agnostic and privacy-preserving framework for 'Subgraph-FL'. By recasting the missing neighbor problem as local feature imputation, FedProp

TABLE I
MAIN NODE CLASSIFICATION RESULTS (AVG CLIENT) AND PERFORMANCE RECOVERY FOR **GCN**. OUR METHOD, **FEDPROP (DIFF-PE)**, USES A DIFFUSION KERNEL. %RECOVERED SHOWS HOW MUCH OF THE PERFORMANCE LOST BY THE ZERO BASELINE IS RECOVERED BY OUR METHOD.

	IID Partition				**Non-IID Partition**			
Metric	**Cora**	**Citeseer**	**Pubmed**	**Ogbn-Arxiv**	**Cora**	**Citeseer**	**Pubmed**	**Ogbn-Arxiv**
Centralized	0.811	0.700	0.785	0.700	0.811	0.700	0.785	0.700
FedProp-Full	0.810	0.700	0.790	0.710	0.810	0.700	0.790	0.690
FedProp (Diff-PE, 1-hop)	0.750	0.660	0.780	0.690	**0.800**	**0.670**	0.760	0.710
FedProp (Diff-PE, 2-hop)	0.770	0.670	0.780	0.670	**0.810**	**0.690**	0.780	0.700
FedProp-Zero	0.620	0.610	0.650	0.600	0.680	0.610	0.650	0.640
% Recovered (1-hop)	**68.4%**	**55.6%**	**92.9%**	**81.8%**	**92.3%**	**66.7%**	**78.6%**	**140.0%**
% Recovered (2-hop)	**78.9%**	**66.7%**	**92.9%**	**63.6%**	**100.0%**	**88.9%**	**92.9%**	**120.0%**

efficiently achieves near-centralized accuracy with zero additional communication cost. Our results confirm it is a practical, highly-efficient solution, offering orders of magnitude better communication-accuracy trade-offs than prior work (Figure 3).

Despite its strengths, FedProp has limitations. Its propagation mechanism, grounded in minimizing Dirichlet energy, is optimized for **homophilic graphs** and may underperform on heterophilic ones. The method also assumes clients possess knowledge of their local **L-hop neighborhood structure**, which might not hold in all FL scenarios. While computationally light for sparse graphs, the local propagation cost could increase for very **dense neighborhoods**. Finally, our evaluation focused on **node classification**, and performance on other tasks is unexplored.

These limitations suggest key directions for **future work**. Developing propagation methods suitable for **heterophilic graphs** is crucial. Investigating FedProp variants that operate with weaker **structural priors** would broaden applicability. Extending the evaluation to **link prediction and graph classification** is needed. Exploring **learnable propagation operators** could allow adaptation to local graph properties or specific GNN backbones, potentially improving performance and robustness further.

REFERENCES

[1] B. Khemani, S. Patil, K. Kotecha, and S. Tanwar, "A review of graph neural networks: concepts, architectures, techniques, challenges, datasets, applications, and future directions," *Journal of Big Data*, vol. 11, no. 1, p. 18, Jan. 2024. [Online]. Available: https://doi.org/10.1186/s40537-023-00876-4

[2] X. Fu, B. Zhang, Y. Dong, C. Chen, and J. Li, "Federated Graph Machine Learning: A Survey of Concepts, Techniques, and Applications," Oct. 2022, arXiv:2207.11812. [Online]. Available: http://arxiv.org/abs/2207.11812

[3] R. Liu, P. Xing, Z. Deng, A. Li, C. Guan, and H. Yu, "Federated Graph Neural Networks: Overview, Techniques and Challenges," Dec. 2022, arXiv:2202.07256. [Online]. Available: http://arxiv.org/abs/2202.07256

[4] L. Peng, N. Wang, N. Dvornek, X. Zhu, and X. Li, "FedNI: Federated Graph Learning With Network Inpainting for Population-Based Disease Prediction," *IEEE Transactions on Medical Imaging*, vol. 42, no. 7, pp. 2032–2043, Jul. 2023. [Online]. Available: https://ieeexplore.ieee.org/document/9815303

[5] Z. Li, M. Bilal, X. Xu, J. Jiang, and Y. Cui, "Federated Learning-Based Cross-Enterprise Recommendation With Graph Neural Networks," *IEEE Transactions on Industrial Informatics*, vol. 19, pp. 673–682, 2023. [Online]. Available: https://consensus.app/papers/federated-learningbased-crossenterprise-bilal-cui/5b0e8fcf9b2e5eca8e0b6ea1ca22e1a0/

[6] X. Li, Y. Zhu, B. Pang, G. Yan, Y. Yan, Z. Li, Z. Wu, W. Zhang, R.-H. Li, and G. Wang, "OpenFGL: A Comprehensive Benchmark for Federated Graph Learning," Jan. 2025, arXiv:2408.16288 [cs]. [Online]. Available: http://arxiv.org/abs/2408.16288

[7] H. Sun, Z. Tu, D. Sui, B. Zhang, and X. Xu, "A Federated Social Recommendation Approach with Enhanced Hypergraph Neural Network," *ACM Trans. Intell. Syst. Technol.*, vol. 16, no. 1, pp. 13:1–13:23, Dec. 2025. [Online]. Available: https://doi.org/10.1145/3665931

[8] Y. Cui, X. Han, J. Chen, X. Zhang, J. Yang, and X. Zhang, "FraudGNN-RL: A Graph Neural Network With Reinforcement Learning for Adaptive Financial Fraud Detection," *IEEE Open Journal of the Computer Society*, vol. 6, pp. 426–437, 2025. [Online]. Available: https://ieeexplore.ieee.org/document/10892045

[9] Y. Yao, W. Jin, S. Ravi, and C. Joe-Wong, "FedGCN: Convergence-Communication Tradeoffs in Federated Training of Graph Convolutional Networks," Dec. 2023, arXiv:2201.12433. [Online]. Available: http://arxiv.org/abs/2201.12433

[10] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," Feb. 2017, arXiv:1609.02907 [cs]. [Online]. Available: http://arxiv.org/abs/1609.02907

[11] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph Attention Networks," Feb. 2018, arXiv:1710.10903 [stat]. [Online]. Available: http://arxiv.org/abs/1710.10903

[12] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," Sep. 2018, arXiv:1706.02216 [cs]. [Online]. Available: http://arxiv.org/abs/1706.02216

[13] C. He, K. Balasubramanian, E. Ceyani, C. Yang, H. Xie, L. Sun, L. He, L. Yang, P. S. Yu, Y. Rong, P. Zhao, J. Huang, M. Annavaram, and S. Avestimehr, "FedGraphNN: A Federated Learning System and Benchmark for Graph Neural Networks," Sep. 2021, arXiv:2104.07145 [cs]. [Online]. Available: http://arxiv.org/abs/2104.07145

[14] K. Zhang, C. Yang, X. Li, L. Sun, and S. M. Yiu, "Subgraph Federated Learning with Missing Neighbor Generation," Nov. 2021, arXiv:2106.13430 [cs]. [Online]. Available: http://arxiv.org/abs/2106.13430

[15] S. Ambekar, Y. Yao, R. Li, and C. Joe-Wong, "FedGAT: A Privacy-Preserving Federated Approximation Algorithm for Graph Attention Networks," Dec. 2024, arXiv:2412.16144 [cs]. [Online]. Available: http://arxiv.org/abs/2412.16144

[16] E. Rossi, H. Kenlay, M. I. Gorinova, B. P. Chamberlain, X. Dong, and M. Bronstein, "On the Unreasonable Effectiveness of Feature propagation in Learning on Graphs with Missing Node Features," May 2022, arXiv:2111.12128. [Online]. Available: http://arxiv.org/abs/2111.12128

[17] J. Gasteiger, A. Bojchevski, and S. Günnemann, "Predict then Propagate: Graph Neural Networks meet Personalized PageRank," Apr. 2022, arXiv:1810.05997 [cs]. [Online]. Available: http://arxiv.org/abs/1810.05997